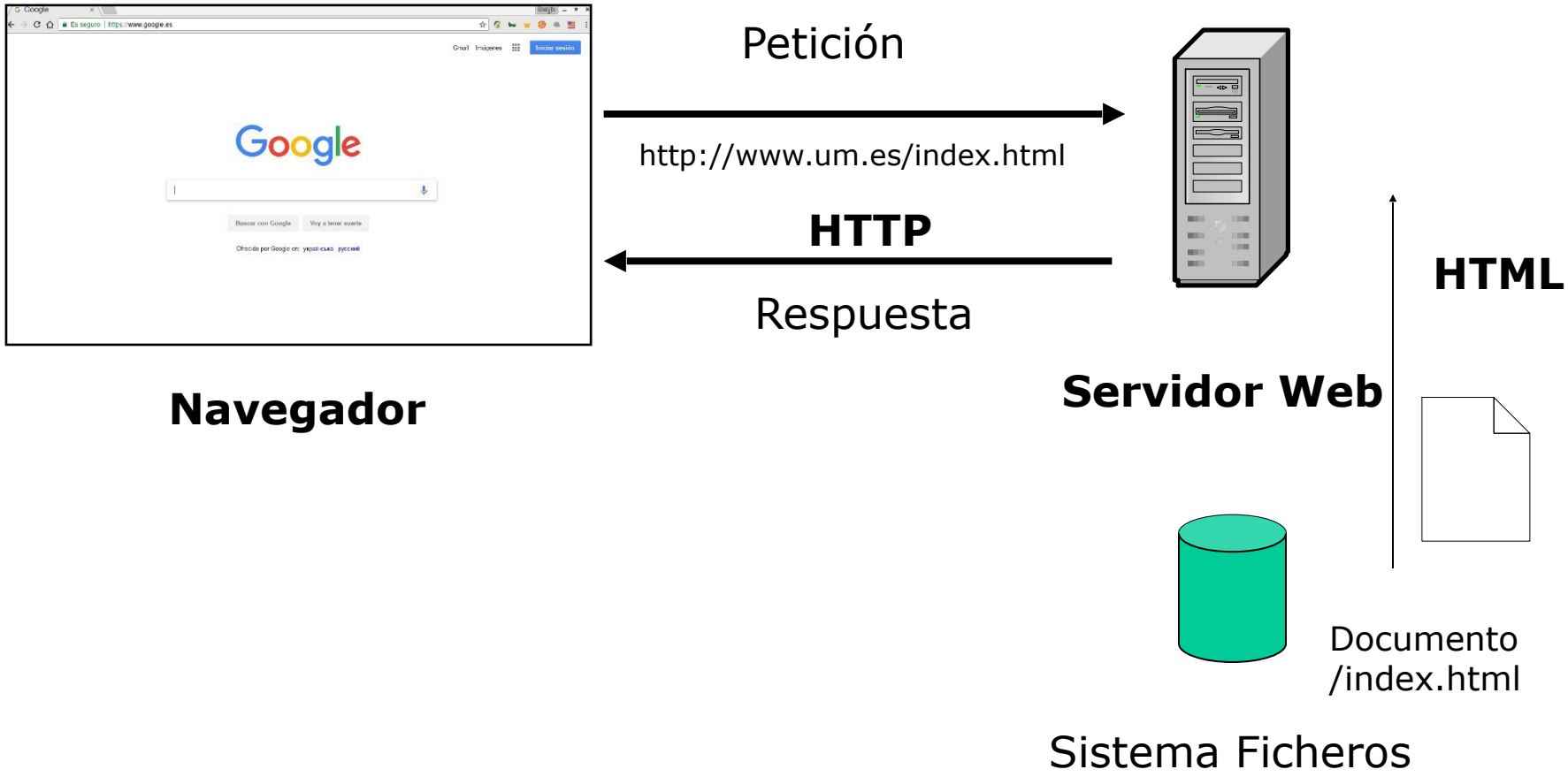
Peticiones asíncronas

Desarrollo de Aplicaciones en Entornos Web
Curso 2024/2025

Contenido

- Intro: Concepto básicos de una aplicación web
  - Protocolo HTTP
- AJAX
  - Ajax – Petición: XMLHttpRequest
  - Ajax – Respuesta : procesamiento de datos
  - Ajax y JSON
- Fetch

Introducción: La Web



Protocolo HTTP

- Protocolo de comunicación entre navegador y servidor web
- Modelo **Petición/Respuesta**
- Es un protocolo **sin estado**:
  - Cada ciclo petición/respuesta es independiente.
- **Tipos de peticiones**:
  - **GET**: petición de un recurso
  - **POST**: envío de datos a un recurso
  - HEAD: petición cabeceras de un recurso
  - ...

Peticiones Asíncronas

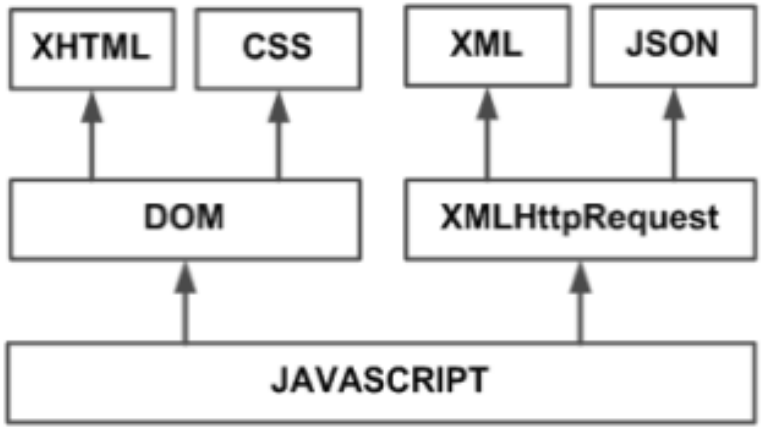
- **Motivación**:
  - En una página de alta, comprobar si existe el identificador de usuario antes de enviar los datos al servidor.
  - En general, **obtener datos del servidor sin necesidad de recargar la página**.

- Se hace necesaria una tecnología que permita el acceso al servidor de forma **asíncrona**
  - **E**n respuesta a eventos de la programación dinámica en HTML.
- **AJAX** (*XMLHttpRequest*) y API **Fetch**
- Supone un **punto de inflexión en el desarrollo de aplicaciones web**.

AJAX

- **AJAX**: ***A****synchronous* ***J****avaScript* ***A****nd* ***X****ML*

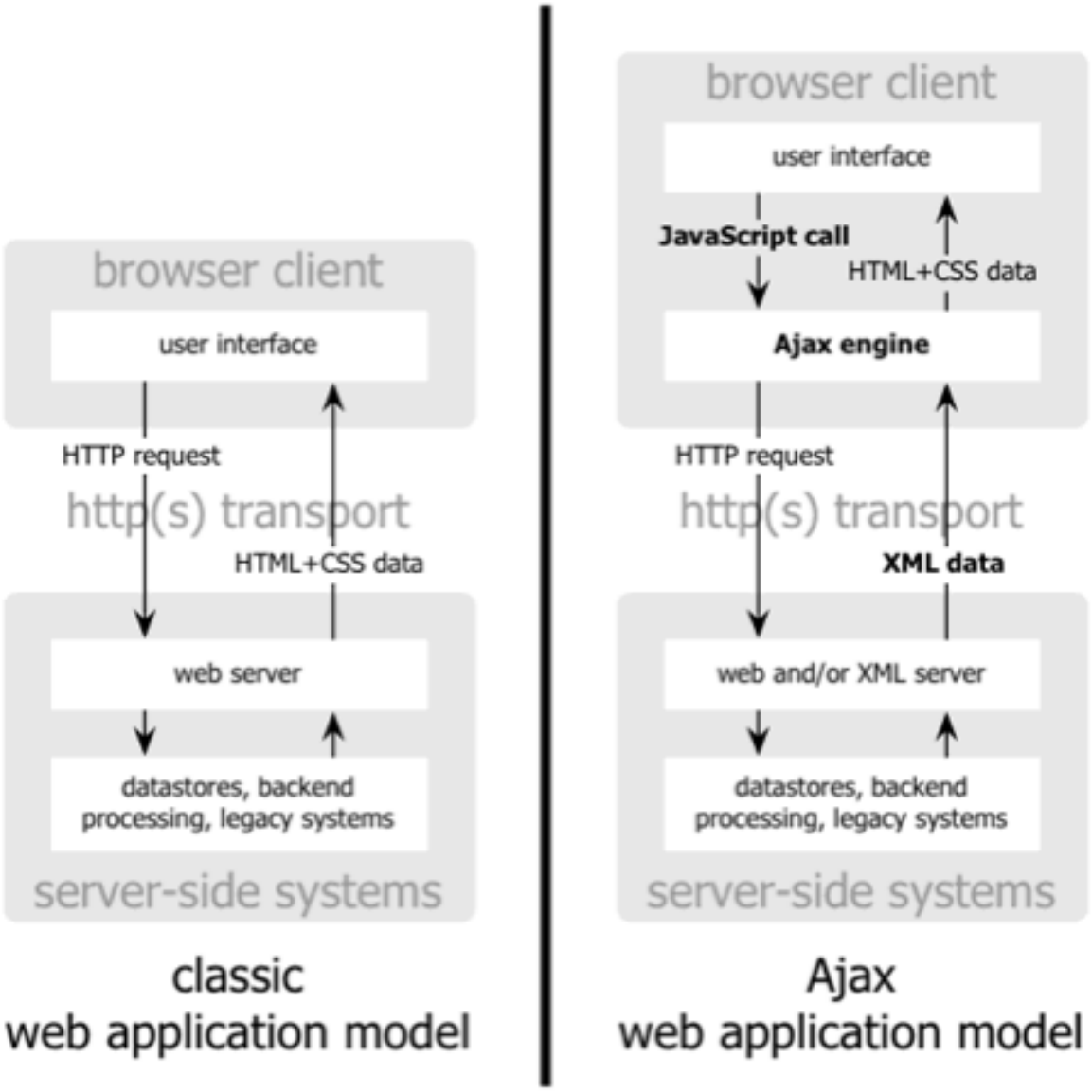
- AJAX representa el soporte de los navegadores para acceder, de modo **asíncrono**, a datos en el servidor utilizando *JavaScript*.



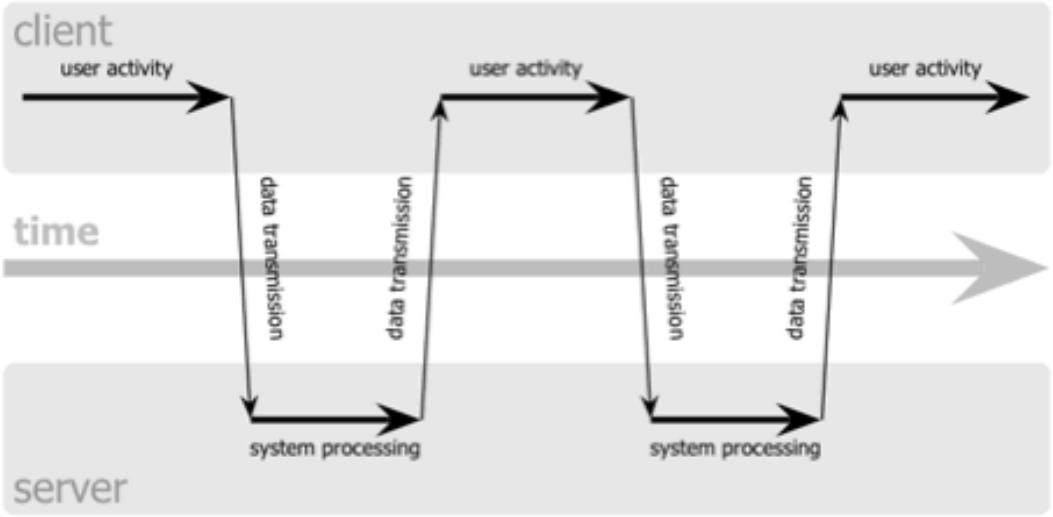
AJAX

- **XMLHttpRequest** es un **objeto JavaScript** que fue diseñado por *Microsoft*
  - adoptado por *Mozilla*, *Apple* y *Google*.
- Actualmente es un *estándar de la W3C*.
- Proporciona una forma fácil de **obtener** información de una **URL sin tener que recargar** la página completa.
  - Una página web puede actualizar sólo una parte de la página **sin interrumpir** lo que el usuario está haciendo.
  - XMLHttpRequest es ampliamente usado en la programación AJAX.
- A pesar de su nombre, XMLHttpRequest puede ser usado para **recibir cualquier tipo de dato**, no solo XML, y admite **otros formatos** además de HTTP (incluyendo **file** y **ftp**).

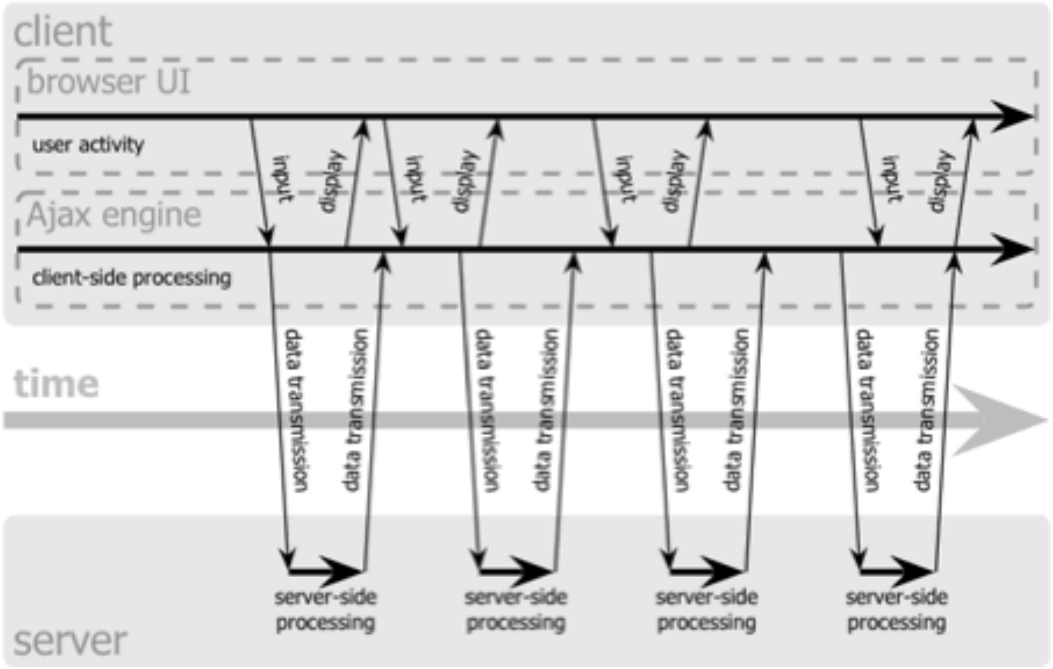
AJAX


browser client
user interface
HTTP request
http(s) transport
HTML+CSS data
web server
datastores, backend processing, legacy systems
server-side systems
classic web application model
browser client
user interface
JavaScript call
HTML+CSS data
Ajax engine
HTTP request
http(s) transport
XML data
web and/or XML server
datastores, backend processing, legacy systems
server-side systems
Ajax web application model

classic web application model (synchronous)



Ajax web application model (asynchronous)



AJAX

- Las peticiones AJAX utilizan un objeto **XMLHttpRequest**.
- Permite realizar con *JavaScript* una petición HTTP al servidor y esperar a que se reciban los datos.
- La clave de la programación con AJAX es la **petición asíncrona de datos al servidor**:
  - Se solicitan datos al servidor, pero el navegador no queda bloqueado a la espera.
  - Cuando llegan los datos, una función *JavaScript* se encarga de procesarlos.

AJAX

- La **función** que recibe los datos del servidor se conoce como **“callback”.**

- La función callback es llamada cada vez que cambia el estado de la petición HTTP:
  - Cuando el **estado de la petición** es *completo* (4) y el **código** de respuesta de **HTTP** es *correcto* (200) se pueden obtener los datos de la petición en texto normal o estructurado en un árbol DOM.

AJAX – Petición HTTP

- **[deprecated]** Los navegadores antiguos no soportan los objetos `XMLHttpRequest`.
- **Solución**:

```
if (typeof XMLHttpRequest != "undefined") {
  req = new XMLHttpRequest();
} else if (window.ActiveXObject) {
  req = new ActiveXObject("Microsoft.XMLHTTP");
}
```

- La **variable** “req” es **global**. Almacena el objeto de la petición y permite a la función callback obtener los datos.

AJAX

- Ejemplo '***Hola Mundo***':

La petición HTTP y la descarga de los contenidos del archivo (mensaje hola mundo) se realizan sin necesidad de recargar la página.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://
www.w3.org/TR/
xhtml1/DTD/xhtml1-transitional.dtd">
<html>
<head>
<title>Hola Mundo con AJAX</title>

<script type="text/javascript">
function descargaArchivo() {
     // Obtener la instancia del objeto XMLHttpRequest
     if(window.XMLHttpRequest) {
          peticion_http = new XMLHttpRequest();
     }
     else if(window.ActiveXObject) {
          peticion_http = new ActiveXObject("Microsoft.XMLHTTP");
     }
     // Preparar la funcion de respuesta
     peticion_http.onreadystatechange = muestraContenido;
     // Realizar peticion HTTP
     peticion_http.open('GET', 'http://localhost/holamundo.txt', true);
     peticion_http.send(null);

     function muestraContenido() {
          if(peticion_http.readyState == 4) {
               if(peticion_http.status == 200) {
                    alert(peticion_http.responseText);
               }
          }
     }
}
window.onload = descargaArchivo;
</script>
</head>
<body></body>
</html>
```

AJAX

- Descripción del ejemplo:
  - instanciar el objeto XMLHttpRequest

    ```
    peticion_http = new XMLHttpRequest();
    ```

  - preparar la **función de respuesta** (calback)

    ```
    peticion_http.onreadystatechange = muestraContenido;
    ```

    - *onreadystatechange* = cuando reciba la respuesta del servidor, se ejecuta manejador del evento.
  - realizar la petición al servidor

    ```
    peticion_http.open('GET', 'http://localhost/prueba.txt', true);
    peticion_http.send(null);
    ```

  - ejecutar la función de respuesta
    - muestraContenido() comprueba que se ha recibido respuesta válida del servidor (propiedad *readyState*)

AJAX - XMLHttpRequest

- Propiedades de ***XMLHttpRequest***

Propiedad	Descripción
readyState	Valor numérico (entero) que almacena el estado de la petición
responseText	El contenido de la respuesta del servidor en forma de cadena de texto
responseXML	El contenido de la respuesta del servidor en formato XML. El objeto devuelto se puede procesar como un objeto DOM
status	El código de estado HTTP devuelto por el servidor (200 para una respuesta correcta, 404 para "No encontrado", 500 para un error de servidor, etc.)
statusText	El código de estado HTTP devuelto por el servidor en forma de cadena de texto: "OK", "Not Found", "Internal Server Error", etc.

AJAX

- Propiedad ***readyState***

Valor	Descripción
0	No inicializado (objeto creado, pero no se ha invocado el método open)
1	Cargando (objeto creado, pero no se ha invocado el método send)
2	Cargado (se ha invocado el método send, pero el servidor aún no ha respondido)
3	Interactivo (se han recibido algunos datos, aunque no se puede emplear la propiedad responseText)
4	Completo (se han recibido todos los datos de la respuesta del servidor)

AJAX

- Métodos de ***XMLHttpRequest***

Método	Descripción
`abort()`	Detiene la petición actual
`getAllResponseHeaders()`	Devuelve una cadena de texto con todas las cabeceras de la respuesta del servidor
`getResponseHeader("cabecera")`	Devuelve una cadena de texto con el contenido de la cabecera solicitada
`onreadystatechange`	Responsable de manejar los eventos que se producen. Se invoca cada vez que se produce un cambio en el estado de la petición HTTP. Normalmente es una referencia a una función JavaScript
`open("metodo", "url")`	Establece los parámetros de la petición que se realiza al servidor. Los parámetros necesarios son el método HTTP empleado y la URL destino (puede indicarse de forma absoluta o relativa)
`send(contenido)`	Realiza la petición HTTP al servidor
`setRequestHeader("cabecera", "valor")`	Permite establecer cabeceras personalizadas en la petición HTTP. Se debe invocar el método `open()` antes que `setRequestHeader()`

AJAX – Petición HTTP

- El segundo paso es **componer la URL del recurso** que procesa la petición. Ejemplo, una **página PHP**.

- **Codificar los parámetros** con la función **encodeURIComponent** teniendo en cuenta que una URL no admite cualquier carácter

```
var url = "validaUsuario.php?id="
        + encodeURIComponent("García");
```

AJAX – Petición HTTP

- El siguiente paso es registrar la URL en el objeto `XMLHttpRequest`:

```
req.open("GET", url, true);
```

- Los **parámetros** son:
  - El **tipo de petición HTTP**: habitualmente GET.
  - La **URL del recurso** que procesa la petición.
  - El **modo de petición**, **asíncrono** (true) o *síncrono* (false).
- Si el modo es asíncrono, se registra la **función** de ***callback*** que es llamada cuando cambie el estado de la petición:

```
req.onreadystatechange = callback;
```

AJAX – Petición HTTP

- Por último, se **realiza la petición**:

  ```
  req.send(null);
  ```

  - El parámetro es el *contenido* de la petición.
  - *contenido*: información que se va a enviar al servidor junto con la petición HTTP (una cadena de texto, un array de bytes o un objeto XML DOM)

- **Importante**: si la llamada es asíncrona, al invocar `send` **el código no espera**. En general, esto es fundamental para que el navegador no se quede bloqueado por la latencia de la comunicación de red.

AJAX – XMLHttpRequest v.2

- En la versión 2 del API, se ofrecen métodos callback específicos para manejar la repuesta en sustitución del manejado de evento original `onreadystatchange`:

  `req.onload;`

  - La función callback es invocada SOLO cuando se ha completado la respuesta, el estado `readyState == 4`.

  `req.onerror;`

  - La función callback es invocada SOLO cuando se ha completado la respuesta y se obtiene un error de comunicación.

- Reemplazan el uso del `onreadystatechange`.

AJAX – XMLHttpRequest v.2

```
// Create an XMLHttpRequest object
const xhttp = new XMLHttpRequest();

// Define a callback function
xhttp.onload = function() {
  // Here you can use the Data
}

// Send a request
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```

AJAX – Respuesta

- El objeto `XMLHttpRequest` controla todo el proceso de petición.
- Conforme **cambia el estado**, notifica a la función de ***callback***.
- La **petición** es **completa** cuando se llega al estado 4.
- Una vez completa, el estado de la **respuesta HTTP** debe ser 200 (**OK**) para que el proceso haya finalizado correctamente.
- En general, el patrón de la **función *callback*** es:

```
if (req.readyState == 4 &&
     req.status == 200) {
          … → procesar la repuesta
```

AJAX – Respuesta

- El objeto `XMLHttpRequest` mantiene el **resultado** de la respuesta de dos modos:
  - Como **texto**, habitualmente formateado en XHTML.

  `req.`**`responseText;`**
  - Como **árbol DOM**:

  `req.`**`responseXML;`**

- El **procesamiento como texto** es el más cómodo para asignar el contenido de la respuesta a un nodo de la página utilizando **`insertAdjacentHTML`**.

AJAX – Envío de datos

- Ejemplo de interacción con el servidor

```
<form>
    <label for="fecha_nacimiento">Fecha de nacimiento:</label>
    <input type="text" id="fecha_nacimiento" name="fecha_nacimiento" />
    <br/>
    <label for="codigo_postal">Codigo postal:</label>
    <input type="text" id="codigo_postal" name="codigo_postal" />
    <br/>
    <label for="telefono">Telefono:</label>
    <input type="text" id="telefono" name="telefono" />
    <br/>
    <input type="button" value="Validar datos" />
</form>
<div id="respuesta"></div>
```

```
var peticion_http = null;
function inicializa_xhr() {
     return new XMLHttpRequest();
}
function crea_query_string() {
     var fecha = document.getElementById("fecha_nacimiento");
     var cp = document.getElementById("codigo_postal");
     var telefono = document.getElementById("telefono");
     return "fecha_nacimiento=" + encodeURIComponent(fecha.value) +
     "&codigo_postal=" + encodeURIComponent(cp.value) +
     "&telefono=" + encodeURIComponent(telefono.value) +
     "&nocache=" + Math.random();
}
```

```
function valida() {
peticion_http = inicializa_xhr();
   if(peticion_http) {
     peticion_http.onreadystatechange = procesaRespuesta;
     peticion_http.open("POST", "http://localhost/validaDatos.jsp", true);
     peticion_http.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
     var query_string = crea_query_string();
     peticion_http.send(query_string);
   }
}
function procesaRespuesta() {
     if(peticion_http.readyState == 4) {
       if(peticion_http.status == 200) {
          document.getElementById("respuesta").insertAdjacentHTML("beforeend",
                                  peticion_http.responseText);
       }
     }
}
```

AJAX

- peticion_http.**setRequestHeader**("Content-Type", "application/x-www-form-urlencoded");
  - *si no* se establece la cabecera Content-Type correcta, el servidor descarta todos los datos enviados mediante el método POST.
  - para enviar parámetros mediante el método **POST**, es obligatorio incluir la cabecera ***Content-Type***

- peticion_http.**send**(query_string);
  - se encarga de enviar los parámetros al servidor (cadena de texto, documento XML,…)
  - Los parámetros se envían en forma de cadena:
    - variables = valores, concatenados mediante **&**
  - *encodeURIComponent() :* imprescindible para evitar problemas con caracteres especiales.

AJAX – Procesamiento de datos

- XMLHttpRequest también permite la recepción de respuestas de servidor en formato XML.
  - *petición_http.**responseXML***
  - Procesamiento mediante ***métodos DOM*** de manejo de documentos XML/HTML

```
<respuesta>
   <mensaje>...</mensaje>
   <parametros>
      <telefono>...</telefono>
      <codigo_postal>...</codigo_postal>
      <fecha_nacimiento>...</fecha_nacimiento>
   </parametros>
</respuesta>
```

AJAX

- Procesamiento de la respuesta

```
function procesaRespuesta() {
   if(peticion_http.readyState == READY_STATE_COMPLETE) {
      if(peticion_http.status == 200) {
     var documento_xml = peticion_http.responseXML;
     var root = documento_xml.getElementsByTagName("respuesta")[0];
     var mensajes = root.getElementsByTagName("mensaje")[0];
     var mensaje = mensajes.firstChild.nodeValue;
     var parametros = root.getElementsByTagName("parametros")[0];
     var telefono = parametros.getElementsByTagName("telefono")[0].firstChild.nodeValue;
     var fecha_nacimiento = parametros.getElementsByTagName("fecha_nacimiento")[0]
                                  .firstChild.nodeValue;
     var codigo_postal = parametros.getElementsByTagName("codigo_postal")
[0].firstChild.nodeValue;

     document.getElementById("respuesta").insertAdjacentHTML("beforeend", mensaje + "<br/>" +
"Fecha nacimiento = " + fecha_nacimiento + "<br/>" + "Codigo postal = " +
               codigo_postal + "<br/>" + "Telefono = " + telefono);
      }
   }
}
```

AJAX y JSON

- Se debe transformar la cadena en un objeto JSON.
  - Es posible el envío y recepción de los parámetros en formato JSON
    - Uso del objeto **JSON** de JS
    - `let resp = xhr.responseText;`
    - `let respJSON = JSON.parse(resp);`
    - `...`
    - `let respText = JSON.stringify(respJSON);`
  - Permite acceder a métodos y propiedades mediante la **notación de puntos** tradicional

    `let fecha_nac =respJSON.parametros.fecha_nac;`

Uso de Promise

- ***Promise*** es la mejor forma para manejar múltiples llamadas en Ajax (asíncronas).
  - se utiliza para evitar problemas con múltiples callbacks y proporcionar una mejor manera de administrar las situaciones de éxito y error.
- *Promise*: **objeto devuelto** por funciones **asincrónicas,** como Ajax.

- Estados de un objeto *Promise*:
  - *pending*: Estado inicial, no terminada exitosamente o rechazada.
  - *fulfilled*: operación exitosa.
  - *rejected*: operación fallida.
  - *settled*: Promise ha sido exitosa o fallida, pero no está pendiente.

Ejemplo Promise XMLHttpRequest es cargar una imagen

```
function imgLoad(url) {
      return new Promise(
                        function(resolve, reject) {
                              var request = new XMLHttpRequest();
                              request.open('GET', url);
                              request.responseType = 'blob';
                              request.onload = function() {
                                    if (request.status === 200 {
                                          resolve(request.response);
                                    } else {
                                      reject(new Error('Image didn\'t load successfully;
                                      error code:' + request.statusText));}
                              };
                              request.onerror = function() {
                                    reject(new Error('There was a network error.'));
                              };
                              request.send(); });
}
```

The XMLHttpRequest **response** property: returns the response's body content as an ***ArrayBuffer, Blob, Document, JavaScript Object,*** *or* ***DOMString***, depending on the value of the request's **responseType** property.

responseType: 'text' , 'json' , 'document'

Dos lugares habituales para usar **Promise**:

- Dentro de una función **async**
  - Se **crea** el objeto de promesa.
  - La **función** asíncrona **devuelve** el objeto de promesa
  - Si async se realiza **correctamente**, el objeto de promesa se resuelve llamando a su método ***resolve***.
  - Si async se realiza con **error**, el objeto de promesa es rechazada llamando a su método ***reject***.
- Fuera de una función **async**
  - Llamar a la función y obtener el objeto de promesa
  - Indicar el manejador de éxito, el manejador de error en el objeto de promesa mediante el método ***then***

```
//Part 1 - Inside Async function
async function makeSomeAsyc(){
  //Create Promise object
  var promiseObj = new Promise(function(resolve, reject){
      ... // Add ajax code here.
      ... // on success, call resolve method, to resolve
      ... // on error, call reject method, to reject
  });
  //Returns Promise object
  return promiseObj;
}
function handleSuccess(data) {   ...   }
function handleError(data) {   ...   }
//Part 2- Outside Async function
var pobj = makeSomeAsyc();
//Attaching success handler to promise
pobj.then(handleSuccess, handleError);
```

Ejemplo Genérico Ajax Call

```
async function makeAjaxCall(url, methodType){
   var promiseObj = new Promise(function(resolve, reject){
      var xhr = new XMLHttpRequest();
      xhr.open(methodType, url, true);
      xhr.send();
      xhr.onreadystatechange = function(){
      if (xhr.readyState === 4){
         if (xhr.status === 200){
            var resp = xhr.responseText;
            var respJSON = JSON.parse(resp);
            resolve(respJSON);
         } else {
            reject(xhr.status);
         }
      }
   }
 });
 return promiseObj;
}
```

Cross Domain (invocaciones)

- Por defecto, AJAX **no permite** hacer llamadas entre diferentes dominios (servidor).
  - El navegador rechaza las llamadas a diferentes dominios

- Para permitir *cross-domains* en las llamadas: **CORS**

CORS

- **Control de acceso HTTP (CORS)**

  - Mecanismo basado en cabeceras HTTP adicionales para permitir que un user agent (navegador) obtenga permiso para acceder a recursos seleccionados desde un servidor, en un **origen distinto (dominio)**

  - Un ejemplo de solicitud de origen cruzado:
    - código JS frontend de una aplicación web que es localizada en `http://domain-a.com` y utiliza *XMLHttpRequest* para cargar otro recurso en `http://api.domain-b.com/data.json`

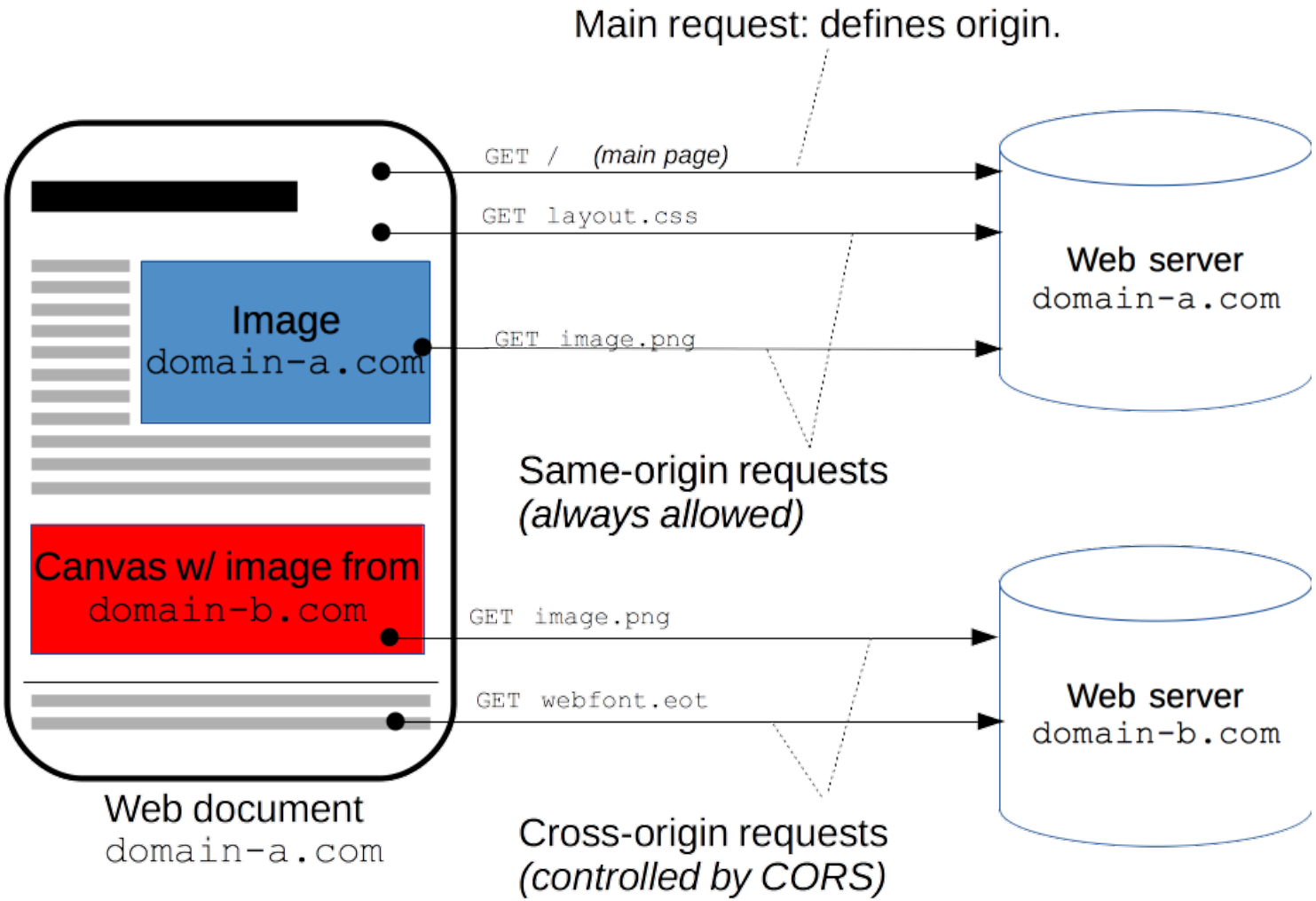
CORS

- Por razones de seguridad, los **navegadores restringen** las solicitudes HTTP de **origen cruzado** iniciadas dentro de un script.

- Por ejemplo:

  - `XMLHttpRequest` y la `API Fetch` siguen la política de **mismo-origen**.

  - Sólo se puede hacer solicitudes HTTP a su propio dominio, salvo que se utilicen cabeceras **CORS**.


Main request: defines origin.
GET / (main page)
GET layout.css
GET image.png
Image
domain-a.com
Web server
domain-a.com
Same-origin requests
(always allowed)
Canvas w/ image from
domain-b.com
GET image.png
GET webfont.eot
Web server
domain-b.com
Web document
domain-a.com
Cross-origin requests
(controlled by CORS)

CORS

- CORS : permite tratar la solicitud AJAX entre multiples dominios.
  - p.e. API de Github están habilitadas para CORS.

- Para habilitar CORS, la respuesta debe contener el encabezado ***Access-Control-Allow-Origin*** con el ***valor de un dominio*** o ****,*** para que funcione para todos.
  - p.e. Github se ha establecido como *

CORS

```
respuesta.setHeader('Access-Control-Allow-Origin', '*');
respuesta.setHeader('Access-Control-Request-Method', '*');
respuesta.setHeader('Access-Control-Allow-Methods', 'OPTIONS, GET');
respuesta.setHeader('Access-Control-Allow-Headers', 'application/json');
respuesta.setHeader('Content-Type', '*');
respuesta.setHeader('status-code',200);
```

CORS - Solicitudes Simples

- Una solicitud de sitio cruzado es aquella que **cumple las siguientes condiciones:**

- Los únicos **métodos** aceptados son:
  - GET
  - HEAD
  - POST.
- Aparte de las cabeceras establecidas automáticamente por user-agent (el navegador, por ej. Connection, User-Agent, etc..), las **únicas cabeceras** que están permitidas para establecer manualmente son:
  - Accept
  - Accept-Language
  - Content-Language
  - Content-Type
- Los únicos valores permitidos de la cabecera **Content-Type** son:
  - application/x-www-form-urlencoded
  - multipart/form-data
  - text/plain

```
var invocation = new XMLHttpRequest();
var url = 'http://bar.other/resources/public-data/';

function callOtherDomain() {
  if(invocation) {
    invocation.open('GET', url, true);
    invocation.onreadystatechange = handler;
    invocation.send();
  }
}
```

```
GET /resources/public-data/ HTTP/1.1
Host: bar.other
User-Agent: Mozilla/5.0 (Macintosh; U; Intel Mac OS X 10.5; en-US; rv:1.9.1b3pre)
Gecko/20081130 Minefield/3.1b3pre
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Connection: keep-alive
Referer: http://foo.example/examples/access-control/simpleXSInvocation.html
Origin: http://foo.example


HTTP/1.1 200 OK
Date: Mon, 01 Dec 2008 00:23:53 GMT
Server: Apache/2.0.61
Access-Control-Allow-Origin: *
Keep-Alive: timeout=2, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: application/xml

[XML Data]
```

CORS - Solicitudes Simples

- Primeras Líneas: la cabecera de solicitud HTTP principal aquí es la cabecera **Origin**
  - muestra que la invocación proviene del contenido en el dominio `http://foo.example`.

- Siguientes Líneas: muestran la respuesta HTTP del servidor en el dominio `http://bar.other`.
  - En respuesta, el servidor envía de vuelta una cabecera `Access-Control-Allow-Origin`.
  - El uso de la cabecera `Origin` y `Access-Control-Allow-Origin` muestran el protocolo de control de acceso en su uso más simple.
  - En este caso, el servidor responde con un `Access-Control-Allow-Origin: *`
    - el recurso puede ser accedido por cualquier dominio en una forma de sitio cruzado.
    - Si se deseara restringir el acceso al recurso solamente para `http://foo.example`, debería devolver lo siguiente:
      - `Access-Control-Allow-Origin: http://foo.example`

API FETCH

- La API Fetch proporciona una interfaz para recuperar recursos (incluso a través de la red).
  - Proporcionado a través del objeto **`window`**
- Similar a XMLHttpRequest, pero con un conjunto de características más potente y flexible.
- Fetch ofrece una definición genérica de los objetos `Request` y `Response` y `Headers`
- Proporciona una definición para conceptos relacionados, como CORS y la semántica de encabezado HTTP origen

API FETCH

- El método **fetch()** toma un argumento obligatorio, la ruta de acceso al recurso que desea recuperar.

- Devuelve una `Promise` que resuelve en `Response` a esa petición, sea o no correcta.

  - También puede pasar opcionalmente un objeto de opciones init como segundo argumento

- Una vez que `Response` es recuperada, hay varios métodos disponibles para definir cuál es el contenido del cuerpo y como se debe manejar

API FETCH (ejemplos de https://davidwalsh.name/fetch)

```
// url (required), options (optional)
fetch('https://davidwalsh.name/some/url', {
    method: 'get'
}).then(function(response) {

}).catch(function(err) {
    // Error :(
});
```

```
// Chaining for more "advanced" handling
fetch('https://davidwalsh.name/some/url').then(function(response) {
    return //...
}).then(function(returnedValue) {
    // ...
}).catch(function(err) {
    // Error :(
});
```

API FETCH - HEADERS

```
// Create an empty Headers instance
var headers = new Headers();

// Add a few headers
headers.append('Content-Type', 'text/plain');
headers.append('X-My-Custom-Header', 'CustomValue');

// Check, get, and set header values
headers.has('Content-Type'); // true
headers.get('Content-Type'); // "text/plain"
headers.set('Content-Type', 'application/json');

// Delete a header
headers.delete('X-My-Custom-Header');

// Add initial values
var headers = new Headers({
    'Content-Type': 'text/plain',
    'X-My-Custom-Header': 'CustomValue'
});
```

API FETCH - HEADERS

- Se puede usar: `append, has, get, set`, and `delete` para modificar las cabeceras de la petición

- Usar las cabeceras en la `Request` :

```
var request = new Request('https://davidwalsh.name/some-url', {
    headers: new Headers({
        'Content-Type': 'text/plain'
    })
});

fetch(request).then(function() { /* handle response */ });
```

API FETCH - REQUEST

- Una instancia de `Request` representa la parte de la solicitud de una llamada de búsqueda.

- Personalizar solicitudes:

  - `method` - `GET`, `POST`, `PUT`, `DELETE`, `HEAD`
  - `url` - URL of the request
  - `headers` - associated `Headers` object
  - `referrer` - referrer of the request
  - `mode` - `cors`, `no-cors`, `same-origin`
  - `credentials` - should cookies go with the request? `omit`, `same-origin`
  - `redirect` - `follow`, `error`, `manual`
  - `integrity` - subresource integrity value
  - `cache` - cache mode (`default`, `reload`, `no-cache`)

API FETCH - REQUEST

```
var request = new Request('https://davidwalsh.name/users.json', {
    method: 'POST',
    mode: 'cors',
    redirect: 'follow',
    headers: new Headers({
        'Content-Type': 'text/plain'
    })
});

// Now use it!
fetch(request).then(function() { /* handle response */ });
```

API FETCH - REQUEST

- Solo se requiere el primer parámetro, la `URL`.
- Cada propiedad pasa a ser de **solo lectura** una vez que se ha creado la instancia de `Request`.
  - Request es un s*tream* y, por lo tanto, debe clonarse cuando se pasa a otra llamada de búsqueda (API de Service Worker).
- La signatura de `fetch()`, sin embargo, actúa como `Request`, por lo que también podría hacer

```
fetch('https://davidwalsh.name/users.json', {
    method: 'POST',
    mode: 'cors',
    redirect: 'follow',
    headers: new Headers({
        'Content-Type': 'text/plain'
    })
}).then(function() { /* handle response */ });
```

API FETCH - RESPONSE

- El método `fetch()` proporciona una instancia de `Response`, pero también se pueden crear objetos `Response` manualmente (*API Service Worker*).

- Con `Response` puedes configurar:

  - `type` - `basic`, `cors`
  - `url`
  - `useFinalURL` - Boolean for if `url` is the final URL
  - `status` - status code (ex: `200`, `404`, etc.)
  - `ok` - Boolean for successful response (status in the range 200-299)
  - `statusText` - status code (ex: `OK`)
  - `headers` - Headers object associated with the response.

API FETCH - RESPONSE

Algunos métodos de `Response`

- `clone()` - Crea un clon de una `Response`.
- `error()` - Devuelve un nuevo objeto `Response` asociado con un error de conexión.
- `redirect()` - Crea una nueva `Response` con una URL diferente.
- `arrayBuffer()` - Devuelve una promesa que se resuelve como un ArrayBuffer.
- `blob()` - Devuelve una promesa que se resuelve como un Blob.
- `json()` - Devuelve una promesa que se resuelve como JSON.
- `text()` - Devuelve una promesa que se resuelve como un texto.

```
// Create your own response for service worker testing
// new Response(BODY, OPTIONS)
var response = new Response('.....', {
    ok: false,
    status: 404,
    url: '/'
});

// The fetch's `then` gets a Response instance back
fetch('https://davidwalsh.name/')
    .then(function(responseObj) {
        console.log('status: ', responseObj.status);
    });
```

API FETCH - RESPONSE ejemplos

Ejemplo de como generar una respuesta en JSON (equivalente a `JSON.parse(string)`):

```
fetch('https://davidwalsh.name/demo/arsenal.json').then(function(response) {
    // Convert to JSON
    return response.json();
}).then(function(j) {
    // Yay, `j` is a JavaScript object
    console.log(j);
});
```

Ejemplo de como generar una respuesta en HTML y BLOB (contenido binario):

```
fetch('/next/page')
  .then(function(response) {
    return response.text();
  }).then(function(text) {
    // <!DOCTYPE ....
    console.log(text);
  });
```

```
fetch('https://davidwalsh.name/flowers.jpg')
    .then(function(response) {
      return response.blob();
    })
    .then(function(imageBlob) {
      document.querySelector('img').src = URL.createObjectURL(imageBlob);
    });
```

API FETCH - envío datos formulario

Ejemplo de envío de datos de Formulario (con FormData o en formato JSON):

```
fetch('https://davidwalsh.name/submit', {
    method: 'post',
    body: new FormData(document.getElementById('comment-form'))
});
```

```
fetch('https://davidwalsh.name/submit-json', {
    method: 'post',
    body: JSON.stringify({
        email: document.getElementById('email').value,
        answer: document.getElementById('answer').value
    })
});
```